SOFTWARE REQUIREMENTS SPECIFICATION

Team Productivity Application

Collaborative Agile Workspace — MVP Build Specification

React | Node.js | PostgreSQL | JWT Auth | Role-Based Access | Sprint Management | Anonymous Retro

TABLE OF CONTENTS

1 System Overview
2 User Roles & Permissions
3 Authentication & User Management
4 Team Management
5 Task Management
6 Sprint Management
7 Kanban Board
8 Retrospective Form
9 Retro Results View
10 Sprint History
11 Navigation Structure
12 Database Schema
13 API Endpoints
14 Acceptance Criteria

1 System Overview

A single-team web application where a Project Manager runs sprint-based task management and developers execute work via a Kanban board. On sprint close, an anonymous retrospective form is automatically unlocked for all team members. The PM views aggregated retro responses alongside completed sprint tasks.

2 User Roles & Permissions

PERMISSION	PROJECT MANAGER	DEVELOPER
Register & create team	✓	✗
Join team via invite code	✗	✓
Create / edit / delete tasks	✓	✗
Move cards on Kanban board	✓	✓

PERMISSION	PROJECT MANAGER	DEVELOPER
Create sprint	✓	✗
Close sprint	✓	✗
Submit retro form	✓	✓
View retro results	✓	✗
View sprint history	✓	✗

3 Authentication & User Management

REGISTRATION

- → Fields: Full name, email, password, role (PM or Developer)
- → PM registration creates the team and generates a unique 6-digit invite code
- → Developer registration requires a valid 6-digit invite code to join a team
- → Password minimum 8 characters, stored as bcrypt hash (salt rounds: 10)
- → No email verification required for MVP

LOGIN & SESSION

- → Email + password authentication
- → Returns a JWT access token with 24-hour expiry
- → Token stored in localStorage on the client
- → All API routes except `/register` and `/login` require a valid JWT in the Authorization header

CONSTRAINTS

- → One account per email address
- → A user can only belong to one team
- → A team can have exactly one PM

4 Team Management

- → PM sees the team invite code on their dashboard (copyable to clipboard)
- → PM sees a list of all team members: name, role, join date
- → No ability to remove members in MVP

5 Task Management

TASK FIELDS

FIELD	TYPE	NOTES
`id`	UUID	Auto-generated
`title`	String (max 100 chars)	Required
`description`	Text	Optional
`assignee_id`	FK → User	Optional; must be a Developer on the team
`priority`	Enum	Low Medium High
`status`	Enum	Backlog, To Do, In Progress, Done
`sprint_id`	FK → Sprint	Null if in backlog
`created_at`	Timestamp	Auto-set

RULES

- → PM creates tasks; they land in **Backlog** status by default
- → PM adds backlog tasks to the active sprint — status becomes **To Do**
- → PM can edit or delete any task that is in Backlog
- → PM cannot delete tasks inside an active sprint
- → Tasks in a closed sprint are read-only

6 Sprint Management

SPRINT FIELDS

FIELD	TYPE	NOTES
`id`	UUID	Auto-generated
`name`	String	e.g. "Sprint 1"
`goal`	Text	Optional one-line goal
`start_date`	Date	Required

FIELD	TYPE	NOTES
`end_date`	Date	Required
`status`	Enum	Active, Completed
`team_id`	FK → Team	Required

RULES

- → Only one sprint can be **Active** at a time per team
- → PM creates a sprint with name, optional goal, start date, and end date
- → PM can add backlog tasks to the active sprint before or during the sprint
- → PM closes the sprint via a "Complete Sprint" button — requires confirmation dialog: *"X tasks are incomplete and will return to backlog. This cannot be undone."*
- → On close: sprint status → Completed; incomplete tasks (To Do / In Progress) reset to Backlog with sprint_id = null; Done tasks remain attached to the sprint; retro form unlocked for all team members
- → A closed sprint cannot be reopened

7 Kanban Board

COLUMNS

To Do → In Progress → Done

BEHAVIOUR

- → Visible only when an active sprint exists
- → All team members (PM + Developers) can drag cards between columns
- → Card displays: task title, assignee initials, priority badge (colour-coded)
- → Clicking a card opens a read-only detail modal; PM sees an Edit button
- → If no active sprint: show message — *"No active sprint. Create one to get started."*
- → Board is locked (drag disabled) once the sprint is marked Completed
- → No real-time sync — manual page refresh reflects updates from other users

8 Retrospective Form

TRIGGER

- → Unlocked for all team members immediately when PM closes the sprint

→ Appears as a banner on the dashboard: *"Sprint [name] is complete. Share your feedback."*

FIXED QUESTIONS

#	QUESTION
1	What went well this sprint?
2	What didn't go well?
3	What should we improve next sprint?

RULES

→ Each team member can submit exactly once per sprint

→ Responses are stored with `sprint_id` and `user_id` — `user_id` is **never** returned in any client-facing API response

→ After submission, form shows: *"Your feedback has been submitted."* Submit button is disabled

→ No editing or deletion of a submitted response

→ PM's submission is treated identically to a Developer's

9 Retro Results View

ACCESS

PM only. Accessible from the sprint history list and from the active completed sprint banner.

LAYOUT

PANEL	CONTENT
Left — Completed Tasks	List of all Done tasks from the sprint. Shows title, assignee, priority badge.
Right — Retro Responses	Responses grouped by question. No names shown. Header shows: *"X of Y team members responded."* PM has a "Close Retro" button to archive the sprint.

10 Sprint History

→ Accessible to PM only via "Past Sprints" in navigation

→ Lists all completed sprints in reverse chronological order

→ Shows per sprint: name, dates, task completion rate (X/Y tasks done), retro response rate

→ Clicking a sprint opens its Retro Results View (read-only, permanently accessible)

11 Navigation Structure

```
/login                          →  Login page
/register                       →  Registration page
/dashboard                      →  Kanban board (if sprint active) or backlog prompt
/backlog                        →  PM: task list + create task + add to sprint
/sprint/new                     →  PM: create sprint form
/retro/:sprintId                →  Submit retro form (all users)
/retro/:sprintId/results        →  PM: view aggregated results
/history                        →  PM: past sprints list
/team                           →  PM: team members + invite code
```

12 Database Schema

```
-- Users
CREATE TABLE users (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name           VARCHAR(100) NOT NULL,
  email          VARCHAR(255) UNIQUE NOT NULL,
  password_hash  TEXT NOT NULL,
  role           VARCHAR(20) NOT NULL CHECK (role IN ('pm', 'developer')),
  team_id        UUID REFERENCES teams(id),
  created_at     TIMESTAMP DEFAULT NOW()
);

-- Teams
CREATE TABLE teams (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name           VARCHAR(100) NOT NULL,
  invite_code    CHAR(6) UNIQUE NOT NULL,
  created_at     TIMESTAMP DEFAULT NOW()
);

-- Sprints
CREATE TABLE sprints (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id        UUID REFERENCES teams(id) NOT NULL,
```

```
  name           VARCHAR(100) NOT NULL,
  goal           TEXT,
  start_date     DATE NOT NULL,
  end_date       DATE NOT NULL,
  status         VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active', 'completed'))
  created_at     TIMESTAMP DEFAULT NOW()
);

-- Tasks
CREATE TABLE tasks (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id        UUID REFERENCES teams(id) NOT NULL,
  sprint_id      UUID REFERENCES sprints(id),
  title          VARCHAR(100) NOT NULL,
  description    TEXT,
  assignee_id    UUID REFERENCES users(id),
  priority       VARCHAR(10) DEFAULT 'medium' CHECK (priority IN ('low', 'medium', 'hig
  status         VARCHAR(20) DEFAULT 'backlog' CHECK (status IN ('backlog', 'todo', 'in
  created_at     TIMESTAMP DEFAULT NOW(),
  updated_at     TIMESTAMP DEFAULT NOW()
);

-- Retro Submissions  (user_id stored for tracking only — never sent to client)
CREATE TABLE retro_submissions (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sprint_id      UUID REFERENCES sprints(id) NOT NULL,
  user_id        UUID REFERENCES users(id) NOT NULL,
  answer_1       TEXT NOT NULL,
  answer_2       TEXT NOT NULL,
  answer_3       TEXT NOT NULL,
  submitted_at   TIMESTAMP DEFAULT NOW(),
  UNIQUE (sprint_id, user_id)
);
```

13 API Endpoints

METHOD	ROUTE	ACCESS	DESCRIPTION
`POST`	`/api/auth/register`	Public	Register user, create/join team
`POST`	`/api/auth/login`	Public	Return JWT

METHOD	ROUTE	ACCESS	DESCRIPTION
`GET`	`/api/team`	All	Team info + members list
`GET`	`/api/team/invite-code`	PM only	Retrieve invite code
`GET`	`/api/sprints/active`	All	Get current active sprint
`GET`	`/api/sprints/history`	PM only	All completed sprints
`POST`	`/api/sprints`	PM only	Create sprint
`PATCH`	`/api/sprints/:id/complete`	PM only	Close sprint, trigger retro
`GET`	`/api/tasks/backlog`	All	All backlog tasks for team
`GET`	`/api/tasks/sprint/:sprintId`	All	Tasks for a given sprint
`POST`	`/api/tasks`	PM only	Create task in backlog
`PATCH`	`/api/tasks/:id`	PM only	Edit task fields
`PATCH`	`/api/tasks/:id/status`	PM + Dev	Move card (update status)
`DELETE`	`/api/tasks/:id`	PM only	Delete backlog task only
`POST`	`/api/retro/:sprintId`	All	Submit retro (once per user)
`GET`	`/api/retro/:sprintId/status`	All	Has current user submitted?
`GET`	`/api/retro/:sprintId/results`	PM only	Aggregated responses, no user_ids
`PATCH`	`/api/retro/:sprintId/close`	PM only	Archive retro

14 Acceptance Criteria — Demo Flow

A complete submission must demonstrate this end-to-end flow without errors:

1. PM registers → team is created → invite code is displayed on dashboard
2. Two developers register using the invite code and appear in the team members list
3. PM creates 4 tasks in the backlog with varying priorities and assignees
4. PM creates a sprint and adds all 4 tasks — board shows all 4 in **To Do**
5. Developer moves 3 tasks to **Done**, leaves 1 in **In Progress**

6. PM clicks "Complete Sprint" → confirmation dialog appears → board locks on confirm
7. All three users see the retro prompt on dashboard and each submit their responses
8. PM opens Retro Results — shows *"3 of 3 responded"*, responses grouped by question, 3 completed tasks on the left panel
9. The 1 incomplete task appears in the backlog (not attached to any sprint)
10. PM visits Past Sprints — closed sprint appears in history with results permanently accessible

— Out of Scope

- ✕ Email notifications or verification
- ✕ Real-time updates / WebSockets
- ✕ Multiple teams per user account
- ✕ Removing team members
- ✕ Sprint reopening
- ✕ Story points or velocity charts
- ✕ Task comments or file attachments
- ✕ Password reset flow
- ✕ Mobile-responsive design
- ✕ Dark mode